Solution based on the NVIDIA RAG Blueprint 2.5.0 customized for a Lenovo ThinkSystem SR650a V4 with 2x NVIDIA RTX PRO 6000 Blackwell GPUs (96 GB each), deployed on top of Ubuntu OS and Canonical k8s (snap).
Replaces Milvus with OpenSearch (snap) as the vector database, assigns workloads across a custom MIG layout, and adds a standalone Triton Inference Server for general-purpose ML inference.

Solution Layout

```
lenovo_nvidia_rag_blueprint/
  config_infra.env           # Shared configuration (hardware, MIG,
OpenSearch)
  setup_infra.sh             # Phase 1: Infrastructure (drivers, K8s, GPU
Operator, OpenSearch)
  deploy_apps.sh             # Phase 2: Applications  (images, Helm chart,
Triton)
  rag_blueprint_lenovo/      # Customized RAG Blueprint (Helm chart, VDB
code, Dockerfiles)
  triton/
    triton-server.yaml       # Standalone Triton deployment (K8s manifests)
```

Component Map

Component	Role	Runs On
NIM LLM (Llama-3.3-Nemotron-Super-49B-v1.5)	Chat completion, answer generation	GPU 0 (full, non-MIG)
NIM Embedding (llama-nemotron-embed-1b-v2)	Query and document embedding	GPU 1 MIG 1g.24 GB
NIM Reranking (llama-nemotron-rerank-1b-v2)	Result re-ranking	GPU 1 MIG 1g.24 GB
NIM OCR (nemoretriever-ocr-v1)	Text extraction from scanned pages	GPU 1 MIG 2g.48 GB
NIM Page Elements (nemotron-page-elements-v3)	Layout detection (tables, charts, text)	GPU 1 MIG 2g.48 GB
NIM Graphic Elements (nemotron-graphic-elements-v1)	Chart and graphic detection	GPU 1 MIG 1g.24 GB

Component	Role	Runs On
NIM Table Structure (nemotron-table-structure-v1)	Table cell/row recognition	GPU 1 MIG 1g.24 GB
RAG Server	Query orchestration (embed, search, rerank, LLM)	CPU
Ingestor Server	Document ingestion API	CPU
nv-ingest	Document processing pipeline (Ray, CPU)	CPU
OpenSearch	Vector store (k-NN) + BM25 full-text search	Bare metal (snap)
Redis	Message broker for nv-ingest	CPU
Triton Inference Server	General ML inference (ONNX, TensorRT, etc.)	GPU 1 MIG 2g.48 GB

GPU Layout

GPU 0 is dedicated entirely to the LLM with no MIG and no time-slicing.
GPU 1 runs MIG mode with three hardware-isolated slices. Within each slice, CUDA time-slicing shares the hardware between multiple workloads.

GPU	Slice	VRAM	Workloads (time-sliced)	K8s Resource
0	Full GPU	96 GB	NIM LLM	`nvidia.com/gpu: 1`
1	1g.24 GB #0 (2x TS)	24 GB	NIM Embed + NIM Rerank	`nvidia.com/mig-1g.24gb: 1`
1	1g.24 GB #1 (2x TS)	24 GB	NIM Graphic Elems + NIM Table Struct	`nvidia.com/mig-1g.24gb: 1`
1	2g.48 GB #0 (3x TS)	48 GB	NIM OCR + NIM Page Elems + Triton	`nvidia.com/mig-2g.48gb: 1`

MIG provides memory and SM isolation between slices. Time-slicing within a slice allows multiple pods to share it by context-switching on the GPU.

What We Customized (vs. Upstream Blueprint)

The upstream NVIDIA RAG Blueprint 2.5.0 ships with Milvus as the vector database and assumes a homogeneous multi-GPU setup. This bundle makes three

categories of changes. **No upstream Helm templates were modified.**

1. OpenSearch Vector Database (replaces Milvus)

Following the NVIDIA "Define Your Own Vector Database" pattern, we created a custom VDB operator under `src/nvidia_rag/utils/vdb/opensearch/`:

File	Purpose
`opensearch_vdb.py`	Full `VectorClient` implementation (index lifecycle, k-NN search, BM25 hybrid, upsert, delete)
`os_queries.py`	OpenSearch query builders (k-NN, hybrid, filter)
`__init__.py`	Package exports

The operator is registered in the VDB factory (`vdb/__init__.py`) so the RAG server and ingestor select it when `VDB_TYPE=opensearch` is set in the Helm values. The `opensearch-py` client library was added to `pyproject.toml` .

OpenSearch itself runs as a **bare-metal snap** (not inside K8s). A K8s Service + Endpoints bridge created by `setup_infra.sh` makes it reachable at `opensearch:9200` from inside the cluster. TLS is enabled with self-signed certificates; the VDB code uses `verify_certs=False` .

Milvus and its MinIO dependency are disabled in `values.yaml` .

2. Custom GPU Resource Assignments

All GPU resource requests in `values.yaml` were changed to match the MIG layout above. The LLM requests `nvidia.com/gpu: 1` , the smaller NIMs request `nvidia.com/mig-1g.24gb: 1` , and the larger NIMs plus Triton request `nvidia.com/mig-2g.48gb: 1` . A custom mig-manager ConfigMap (`mig-config-lenovo-2gpu.yaml`) tells the GPU Operator how to slice GPU 1.

3. Custom Container Images

The stock NVIDIA images do not contain the OpenSearch VDB code. Two custom images are built during deployment with `buildah` :

- `lenovo/rag-server:2.5.0-opensearch`
- `lenovo/ingestor-server:2.5.0-opensearch`

These are built from the existing Dockerfiles, injected into the containerd image store, and referenced in `values.yaml`. No pre-built tarballs are shipped; images are always built fresh on the target machine.

4. Triton Inference Server (standalone)

A separate `triton/triton-server.yaml` deploys Triton 26.03 into its own `triton` namespace. It shares the 2g.48 GB MIG slice with OCR and Page Elements via time-slicing. An initContainer pre-loads the official densenet_onnx quickstart model for immediate smoke-testing. Additional models can be copied into the `triton-model-repo` PVC at any time.

Support Matrix

Component	Version	Provider
Ubuntu	24.04 LTS (Ubuntu Pro)	Canonical
Canonical K8s (snap)	1.32	Canonical
OpenSearch (snap)	2/stable	Canonical
NVIDIA Drivers (open kernel)	595.58.03	NVIDIA
CUDA Toolkit	13	NVIDIA
GPU Operator	v26.3.0	NVIDIA
NVIDIA RAG Blueprint	2.5.0	NVIDIA
NIM Operator	3.1.0	NVIDIA
NIM Containers	2.0.x	NVIDIA
Triton Inference Server	26.03	NVIDIA

Hardware Requirements

Resource	Minimum	Recommended
Server	2U rack, PCIe Gen5 x16 slots	Lenovo ThinkSystem SR650a V4
GPUs	2x NVIDIA RTX PRO 6000 Blackwell (96 GB)	Same
RAM	512 GB	1 TB

Resource	Minimum	Recommended
CPU	64 cores	192 cores
Storage	1 TB NVMe	2 TB NVMe

Installation

Prerequisites

1. Ubuntu 24.04 LTS installed on the server
2. Root (sudo) access
3. Internet connectivity (for container image pulls and model downloads)
4. An NGC API key with access to NIM containers on `nvcr.io`

Step 1 -- Deploy Infrastructure

`setup_infra.sh` installs NVIDIA drivers, OpenSearch, Canonical K8s, the GPU Operator with MIG configuration, and creates the OpenSearch K8s bridge.

```
sudo bash setup_infra.sh
```

The script will prompt for a **reboot** after installing NVIDIA drivers on the first run. After rebooting, re-run the same command to continue from where it left off. The script is idempotent.

Phases:

#	What it does
1	NVIDIA drivers (open kernel 595.58.03) + CUDA 13
2	OpenSearch snap (TLS, admin password, k-NN plugin)
3	Canonical K8s 1.32 (single-node, DNS, local-storage, LB)
4	GPU Operator v26.3.0 + MIG configuration (mig-manager)
5	Load balancer (MetalLB / k8sd, L2 mode)
6	K8s namespaces, OpenSearch Service+Endpoints bridge, secrets

Step 2 -- Deploy Applications

`deploy_apps.sh` builds the custom images, installs the NIM Operator, deploys the RAG Blueprint via Helm, and deploys Triton.

```
sudo bash deploy_apps.sh
```

The script will prompt for the NGC API key if not already saved. On subsequent runs it detects the existing deployment and offers to upgrade.

Phases:

#	What it does
1	Validate prerequisites (K8s, GPU Operator, OpenSearch bridge)
2	NGC credentials (prompt or load from store)
3	Build custom images with buildah (OpenSearch VDB support)
4	Build Helm chart dependencies
5	Install NIM Operator (CRDs + controller)
6	Helm install/upgrade of the RAG Blueprint
7	Deploy Triton Inference Server into `triton` namespace
8	Post-deploy status and pod readiness check

Flags:

```
--upgrade    Upgrade an existing Helm release
--dry-run    Render templates only (helm template)
--uninstall  Remove the RAG Blueprint and Triton deployments
```

Step 3 -- Wait for NIM Images

NIM container images are large (10--40 GB each). After `deploy_apps.sh` completes, pods will be in `ContainerCreating` or `Pending` state while images are pulled. Monitor progress:

```
k8s kubectl get pods -n rag -w
k8s kubectl get pods -n triton -w
```

Step 4 -- Access the Frontend

Once all pods are running, expose the RAG frontend:

```
k8s kubectl port-forward -n rag svc/rag-frontend 3000:3000 --address 0.0.0.0
&
```

Then open `http://<server-ip>:3000` in a browser.

Verify Triton

```
# Check Triton readiness
k8s kubectl exec -n triton deploy/triton-inference-server -- \
  curl -s localhost:8000/v2/health/ready

# List loaded models
k8s kubectl exec -n triton deploy/triton-inference-server -- \
  curl -s localhost:8000/v2/models | python3 -m json.tool

# Test inference with the SDK container (from a machine with Docker):
docker run --rm --net=host nvcr.io/nvidia/tritonserver:26.03-py3-sdk \
  /workspace/install/bin/image_client -m densenet_onnx -c 3 -s INCEPTION \
  -u <server-ip>:8000 /workspace/images/mug.jpg
```

Adding Models to Triton

Copy model directories into the PVC:

```
k8s kubectl cp ./my-model triton/<triton-pod>:/models/my-model
```

Triton will detect and load new models automatically (polling mode is enabled).